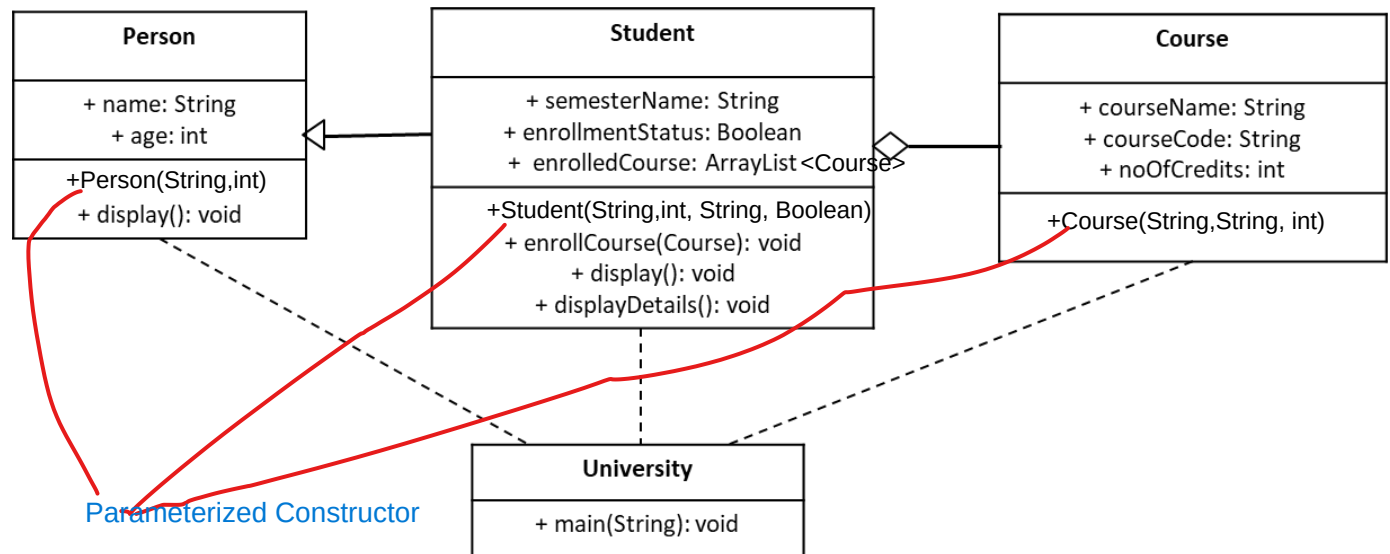


1. Inheritance+Association(Unidirectional):

A university wants to create an automated system to manage courses, students. The Person class is the base class with name, age, and display() method, from which the Student is derived and overrides the method. The Student class has additional attributes like semesterName, enrollmentStatus, listOfEnrolledCourse, and methods to enroll in courses and displayDetails(). The Course class represents a course and contains details like courseName, courseCode, noOfCredit.

UML diagram:



Code:

```
import java.util.ArrayList;
```

```
class Person {
    public String name;
    public int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

```
public void display() {  
    System.out.println("Name: " + name + "\nAge: " + age);  
}  
}
```

```
class Course {  
    public String courseName;  
    public String courseCode;  
    public int noOfCredits;  
  
    public Course(String courseName, String courseCode, int noOfCredits) {  
        this.courseName = courseName;  
        this.courseCode = courseCode;  
        this.noOfCredits = noOfCredits;  
    }  
}
```

```
class Student extends Person {  
    public String semesterName;  
    public boolean enrollmentStatus;  
    public ArrayList<Course> enrolledCourses;  
  
    public Student(String name, int age, String semesterName, boolean enrollmentStatus) {  
        super(name, age);  
        this.semesterName = semesterName;  
        this.enrollmentStatus = enrollmentStatus;  
    }  
}
```

```
    this.enrolledCourses = new ArrayList<>();  
}
```

```
public void enrollCourse(Course course) {  
    if (enrollmentStatus) {  
        enrolledCourses.add(course);  
        System.out.println("Enrolled in " + course.courseName);  
    } else {  
        System.out.println("Cannot enroll, student is not active.");  
    }  
}
```

@Override

```
public void display() {  
    super.display();  
    System.out.println("Semester: " + semesterName + "\nEnrollment Status: " + enrollmentStatus);  
  
    for (Course course : enrolledCourses) {  
        System.out.println(course.courseName);  
    }  
}
```

```
public void displayDetails() {  
  
    System.out.println("Enrolled Courses: " );  
    for (Course course : enrolledCourses) {  
        System.out.println("Course name: " + course.courseName + ", Course code: " +  
course.courseCode);  
    }  
}
```

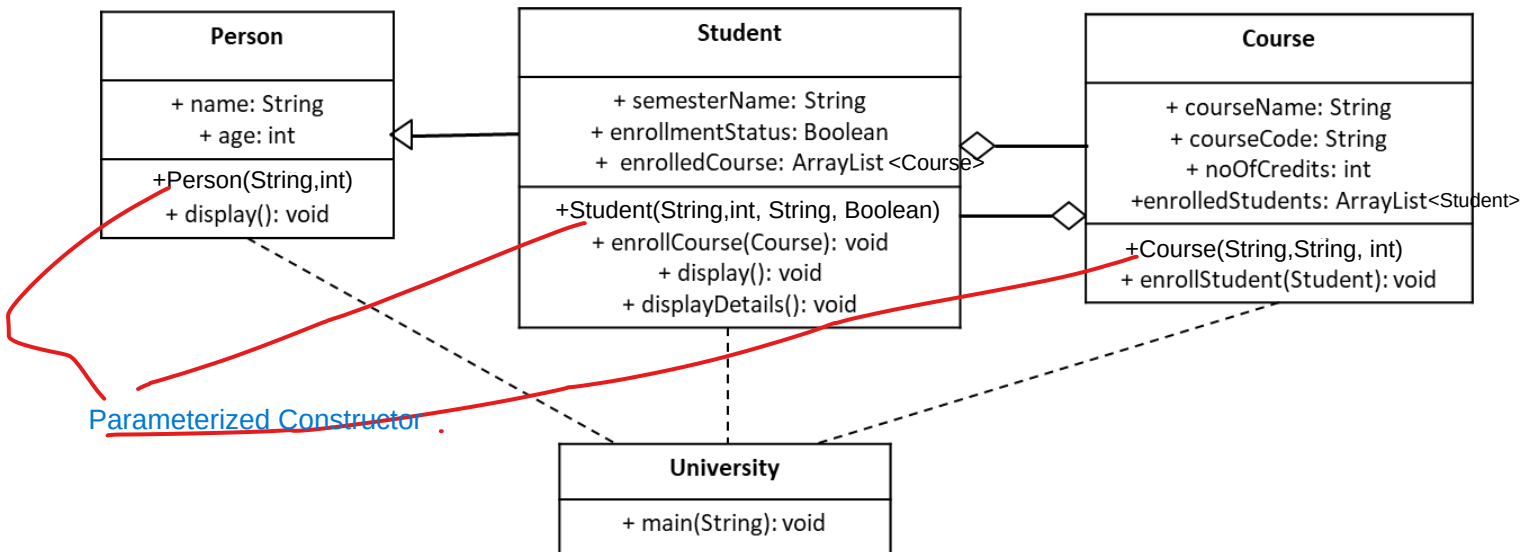
```
}  
}
```

```
public class University {  
    public static void main(String[] args) {  
        Course c1 = new Course("Mathematics", "MATH101", 3);  
        Course c2 = new Course("Physics", "PHYS101", 4);  
  
        Student s1 = new Student("John", 20, "Spring 2025", true);  
        s1.enrollCourse(c1);  
        s1.enrollCourse(c2);  
        s1.display();  
        s1.displayDetails();  
    }  
}
```

2. Inheritance+Association(Bidirectional):

A university wants to create an automated system to manage courses, students. The Person class is the base class with name, age, and display() method, from which the Student is derived and overrides the method. The Student class has additional attributes like semesterName, enrollmentStatus, listOfEnrooledCourse, and methods to enroll in courses and displayDetails(). The Course class represents a course and contains details like courseName, courseCode, noOfCredit and a list of enrolled students.

UML diagram:



Code:

```
import java.util.ArrayList;
```

```
class Person {
    public String name;
    public int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

```
public void display() {  
    System.out.println("Name: " + name + "\nAge: " + age);  
}  
}
```

```
class Course {  
    public String courseName;  
    public String courseCode;  
    public int noOfCredits;  
    public ArrayList<Student> enrolledStudents;  
  
    public Course(String courseName, String courseCode, int noOfCredits) {  
        this.courseName = courseName;  
        this.courseCode = courseCode;  
        this.noOfCredits = noOfCredits;  
        this.enrolledStudents = new ArrayList<>();  
    }  
  
    public void enrollStudent(Student student) {  
        if (!enrolledStudents.contains(student)) {  
            enrolledStudents.add(student);  
        }  
    }  
}
```

```
class Student extends Person {
```

```
public String semesterName;

public boolean enrollmentStatus;

public ArrayList<Course> enrolledCourses;
```

```
public Student(String name, int age, String semesterName, boolean enrollmentStatus) {
    super(name, age);
    this.semesterName = semesterName;
    this.enrollmentStatus = enrollmentStatus;
    this.enrolledCourses = new ArrayList<>();
}
```

```
public void enrollCourse(Course course) {
    if (enrollmentStatus) {
        enrolledCourses.add(course);
        course.enrollStudent(this);
        System.out.println("Enrolled in " + course.courseName);
    }
    else {
        System.out.println("Cannot enroll, student is not active.");
    }
}
```

```
@Override
```

```
public void display() {
    super.display();
    System.out.println("Semester: " + semesterName + "\nEnrollment Status: " + enrollmentStatus);
}
```

```
public void displayDetails() {
```

```
        System.out.println("Enrolled courses: ");  
        for (Course course : enrolledCourses) {  
            System.out.println("Course name: " + course.courseName + ", Course code: " +  
course.courseCode);  
        }  
    }  
}
```

```
public class University {  
    public static void main(String[] args) {  
        Course c1 = new Course("Mathematics", "MATH101", 3);  
        Course c2 = new Course("Physics", "PHYS101", 4);  
  
        Student s1 = new Student("John", 20, "Spring 2025", true);  
        Student s2 = new Student("Ruby", 22, "Spring 2025", true);  
  
        s1.enrollCourse(c1);  
        s1.enrollCourse(c2);  
        s2.enrollCourse(c1);  
  
        s1.display();  
        s1.displayDetails();  
        s2.display();  
        s2.displayDetails();  
  
    }  
}
```